CS 240 - Lab 8

TAs: Malay

Spring 2025

Welcome to the 8th lab of this course!! phew

Instructions:

- This lab focuses on understanding CNNs, including the architectures used in SOTA image detection tasks. Hope you have fun with the lab!!
- All the questions are ungraded.

1 Implementing Convolution and Pooling from Scratch (NumPy)

In this exercise, the aim is to implement the forward pass of 2D convolution and pooling operations manually using NumPy. You will also apply different kernels on images to understand what convolutional layers actually compute.

1.1 Implement 2D Convolution (Forward Pass)

Complete a function `convolve2d(image, kernel, stride, padding)` that performs convolution on a 2D grayscale image. Your implementation should support:

- Both Zero and Non-zero padding
- Arbitrary stride
- Multiple output channels (when kernel is 3D: height × width × out_channels)

Read the definitions of stride and padding online.
Note: You may use nested loops for clarity. You can also read about `Img2vec` for vectorized implementation.

1.2 Implement Max Pooling and Average Pooling

Complete two functions : `max_pool2d(image, pool_size, stride)` and `avg_pool2d(image, pool_size, stride)`.

1.3 Apply Hand-crafted Kernels

Apply the following kernels (already provided in the template code) on at least three different images (one with strong edges, one smooth, one textured):

- Identity kernel
- Edge detection (Laplacian)
- Sobel Horizontal & Vertical
- Sharpen
- Gaussian blur (3×3)
- Emboss

For each kernel, observer:

- Original image vs Convolved output
- Convolved → ReLU → MaxPool output

Analysis Questions

1. Why does the Laplacian kernel highlight edges?
2. What is the effect of increasing stride in convolution? In pooling?
3. How does max pooling help in achieving translation invariance?
4. What would happen if we remove padding when using large kernels?

2 Training Deep CNNs on CIFAR-10: Why Architecture Matters

In this exercise, we will start with a simple shallow CNN and progressively improve it by adding Batch Normalization, increasing depth, and introducing skip connections. Observe how each change affects training speed, stability, and accuracy. You will be using PyTorch to solve this lab.

2.1 Dataset: CIFAR-10

CIFAR-10 is a dataset of 60,000 images each of size 32×32×3 (RGB). It has images belonging to 10 classes: airplane, car, bird, cat, deer, dog, frog, horse, ship, truck. It is a relatively harder dataset compared to say MNIST, and for successful classification, your network must efficiently extract useful features from very little data per image.

2.2 Important building blocks - definitions and intuition

Convolutional layer (Conv2d)

Slides a small learnable filter (kernel) across the image and computes dot products.
Intuition: detects local patterns (edges, textures, corners, etc). Different filters learn different patterns. The deeper we go, the more complex and abstract the patterns become.

Batch Normalization (BatchNorm2d) [2]

Normalizes the activations of each mini-batch (subtract mean, divide by std, then learn scale and shift). Usually placed **after** convolution and **before** activation.
Intuition: makes training more stable and faster — reduces internal covariate shift, allows higher learning rates, acts like a weak regularizer.
To know more, please read Section 14.2.4 in the prescribed PML textbook.

ReLU activation

$f(x) = \max(0, x)$
Intuition: introduces non-linearity, helps with vanishing gradient problem (compared to sigmoid/tanh), very fast to compute.

Max Pooling

Takes the maximum value in each small window (e.g. 2×2), reduces spatial dimensions.
Intuition: provides translation invariance (small shifts in input → same output) + reduces number of parameters + computational cost.

Dropout

Randomly sets activations to zero during training.
Intuition: prevents neurons from "co-adapting", forcing redundancy and leading to better generalisation.

Residual / Skip connection

In a residual block we compute $y = F(x) + x$ (instead of $y = F(x)$).
Intuition: allows gradients to flow directly through the identity path → dramatically easier to train very deep networks (tens or hundreds of layers) without degradation.

Global Average Pooling

Instead of flattening large feature maps → average each channel spatially → produces one value per channel.

Intuition: much fewer parameters than flattening + large FC layer, reduces overfitting, forces the network to learn spatially distributed features.

2.3 How to think about CNN training

When training, three things matter:

1. **Representation power** Can the network represent the function?
2. **Optimization** Can gradient descent actually find good parameters?
3. **Generalization** Does it work on unseen data?

Each architectural change targets one of these.

2.4 Choosing Optimizers and Learning Rate

This lab focuses on comparing architectures, so you should use a consistent and simple optimization strategy rather than extensive tuning.

Default choice. Start all experiments with **Adam**, with a learning rate of 10^{-3}. This works reliably across all models and provides a fair baseline.

Switch to **SGD with momentum** if:

- Training is stable but final accuracy is lower than expected
- You want better generalization (especially for deeper models)

Use:

- LR $= 0.01$ (or 0.1 with BatchNorm), momentum $= 0.9$

Adjusting the learning rate.

- Loss oscillates or diverges $\rightarrow$ decrease LR ($\times 0.1$)
- Loss decreases very slowly $\rightarrow$ increase LR ($\times 2$–10)
- Training plateaus early $\rightarrow$ reduce LR during training

2.5 Tasks

You are required to do the following tasks using PyTorch. We recommend using Kaggle or Colab to enable GPU acceleration.

1. Load and preprocess the CIFAR-10 dataset.
    - Normalize images (important for stable training)
    - Use data augmentation: Do Random cropping and Horizontal flipping.
2. Train the following models **progressively** and record test accuracy, training curves, and observations:

(a) **Model 1: Baseline Shallow CNN**
Structure:
Conv → ReLU → Pool
Conv → ReLU → Pool
FC → Softmax

(b) **Model 2: Baseline + Batch Normalization**
Add `BatchNorm2d` after every convolution.

(c) **Model 3: Deeper CNN (without skip connections) [3]**
Increase depth to 5 to 6 convolutional blocks.
(Refer to Section 13.4.4 in PML textbook for more details.)

(d) **Model 4: Deep CNN with Residual (Skip) Connections [1]**
Implement a simple Residual Block and build a ResNet-style network.

3. For each model, plot:

   - Training and Validation Loss curves
   - Training and Validation Accuracy curves

   Look for Overfitting (training error very low, validation error high), Underfitting (both errors low) and Optimization issues (Loss not decreasing smoothly)

4. Compare all four models and answer:

   - How much does Batch Normalization help?
   - Why does a plain deeper network sometimes perform worse?
   - How do skip connections help in training deeper networks?

References

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[2] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.

[3] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.